HWK#3

Shreesh Karjagi

Problem 1

We need to show that if $x_1 \sim U(0,1)$ and $x_2 \sim U(0,1)$, then the transformation

$$y_1 = \sqrt{-2\ln x_1}\cos(2\pi x_2), \qquad y_2 = \sqrt{-2\ln x_1}\sin(2\pi x_2)$$

produces $p(y_1, y_2) = \mathcal{N}(y_1 \mid 0,1)\,\mathcal{N}(y_2 \mid 0,1)$.

Proof. We use the change of variables formula. Since $p(x_1, x_2) = 1$ on $[0,1]^2$:

$$p(y_1, y_2) = p(x_1, x_2)\left|\det \frac{\partial(x_1, x_2)}{\partial(y_1, y_2)}\right|$$

It helps to switch to polar coordinates. Let $r = \sqrt{y_1^2 + y_2^2}$ and $\theta = \operatorname{atan2}(y_2, y_1)$, so:

$$y_1 = r\cos\theta, \quad y_2 = r\sin\theta$$

From the transformation: $r = \sqrt{-2\ln x_1}$ and $\theta = 2\pi x_2$, which gives us the inverse:

$$x_1 = e^{-r^2/2}, \qquad x_2 = \frac{\theta}{2\pi}$$

The Jacobian of (x_1, x_2) with respect to (r, θ) is:

$$\frac{\partial x_1}{\partial r} = -r\,e^{-r^2/2}, \quad \frac{\partial x_1}{\partial \theta} = 0, \quad \frac{\partial x_2}{\partial r} = 0, \quad \frac{\partial x_2}{\partial \theta} = \frac{1}{2\pi}$$

So $\left|\det J_{(x_1,x_2)/(r,\theta)}\right| = \frac{r}{2\pi}e^{-r^2/2}$.
Since the Jacobian from (y_1, y_2) to (r, θ) contributes $|dy_1\,dy_2| = r\,dr\,d\theta$:

$$p(y_1, y_2)\,dy_1\,dy_2 = 1 \cdot \frac{r}{2\pi}e^{-r^2/2}\,dr\,d\theta = \frac{1}{2\pi}e^{-r^2/2}\,r\,dr\,d\theta$$

Converting back: $r\,dr\,d\theta = dy_1\,dy_2$, so:

$$\boxed{p(y_1, y_2) = \frac{1}{2\pi}e^{-(y_1^2+y_2^2)/2} = \mathcal{N}(y_1 \mid 0,1)\,\mathcal{N}(y_2 \mid 0,1)}$$

Algorithm: To sample from a univariate normal, draw $x_1, x_2 \sim U(0,1)$ independently and compute $y_1 = \sqrt{-2\ln x_1}\cos(2\pi x_2)$. Then $y_1 \sim \mathcal{N}(0,1)$. (We can also use $y_2 = \sqrt{-2\ln x_1}\sin(2\pi x_2)$ as a second independent sample.

Problem 2

Given a symmetric Gaussian proposal $\tilde{q}(\mathbf{x}'|\mathbf{x}) = \mathcal{N}(\mathbf{x}' \mid \mathbf{x}, \sigma_q^2\mathbf{I})$ and target $p(\mathbf{x}) = \mathcal{N}(\mathbf{x} \mid \mathbf{0}, \sigma_p^2\mathbf{I})$ with $\dim(\mathbf{x}) = N$.

We want to show:

$$\left\langle \log \frac{p(\mathbf{x}')}{p(\mathbf{x})} \right\rangle_{\tilde{q}(\mathbf{x}'|\mathbf{x})} = -\frac{N\sigma_q^2}{2\sigma_p^2}$$

Proof. Since $p(\mathbf{x}) = \mathcal{N}(\mathbf{0}, \sigma_p^2\mathbf{I})$:

$$\log \frac{p(\mathbf{x}')}{p(\mathbf{x})} = -\frac{1}{2\sigma_p^2}\left(\|\mathbf{x}'\|^2 - \|\mathbf{x}\|^2\right)$$

Under the proposal, $\mathbf{x}' = \mathbf{x} + \boldsymbol{\epsilon}$ where $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \sigma_q^2\mathbf{I})$ is independent of the randomness in $\mathbf{x}$. So:

$$\|\mathbf{x}'\|^2 = \|\mathbf{x} + \boldsymbol{\epsilon}\|^2 = \|\mathbf{x}\|^2 + 2\mathbf{x}^T\boldsymbol{\epsilon} + \|\boldsymbol{\epsilon}\|^2$$

Taking the expectation over $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \sigma_q^2\mathbf{I})$ (with $\mathbf{x}$ held fixed):

$$\begin{aligned}\mathbb{E}_{\boldsymbol{\epsilon}}\left[\|\mathbf{x}'\|^2 - \|\mathbf{x}\|^2\right] &= \mathbb{E}[2\mathbf{x}^T\boldsymbol{\epsilon}] + \mathbb{E}[\|\boldsymbol{\epsilon}\|^2] \\ &= 0 + N\sigma_q^2\end{aligned}$$

since $\mathbb{E}[\boldsymbol{\epsilon}] = \mathbf{0}$ and $\mathbb{E}[\|\boldsymbol{\epsilon}\|^2] = N\sigma_q^2$. Therefore:

$$\boxed{\left\langle \log \frac{p(\mathbf{x}')}{p(\mathbf{x})} \right\rangle_{\tilde{q}(\mathbf{x}'|\mathbf{x})} = -\frac{N\sigma_q^2}{2\sigma_p^2} \qquad \square}$$

Discussion. This result says the expected log-ratio is negative and grows linearly with dimension N. In high dimensions the proposed state $\mathbf{x}'$ will almost certainly have lower probability than $\mathbf{x}$, since adding noise $\boldsymbol{\epsilon}$ increases $\|\mathbf{x}'\|^2$ on average which would be pushing the proposed point further from the mode.

The MH acceptance probability involves $\min(1, p(\mathbf{x}')/p(\mathbf{x}))$. Since the ratio $p(\mathbf{x}')/p(\mathbf{x})$ is typically $\exp(-N\sigma_q^2/(2\sigma_p^2))$ which decays exponentially in N, acceptance rates drop to near zero for fixed σ_q as $N \to \infty$.

To maintain a reasonable acceptance rate we need $\sigma_q \sim 1/\sqrt{N}$ but then the step size shrinks and the chain mixes very slowly. This is the fundamental issue of dimensionality for random-walk MH and the chain either rejects too often (large σ_q) or moves too slowly (small σ_q).

Problem 3

We have a binary Ising MRF on a 7×7 lattice:

$$p(\mathbf{x}) \propto \exp\left(\sum_{i \sim j} \mathbb{I}[x_i = x_j]\right)$$

Nodes labeled 1–49 (row-major). Even-indexed nodes 2–24 are observed as $x_i = 1$, and even-indexed nodes 26–48 are observed as $x_i = 0$. We must sample the 25 odd-indexed (unobserved) nodes conditioned on this evidence.

Part (i): Sampling method

The simplest method is **Gibbs sampling**. We sweep through all unobserved (odd-indexed) nodes one at a time, sampling each from its full conditional distribution given its neighbors (which may be observed or other unobserved nodes at their current values).

For unobserved node x_i with neighbors $\mathcal{N}(i)$:

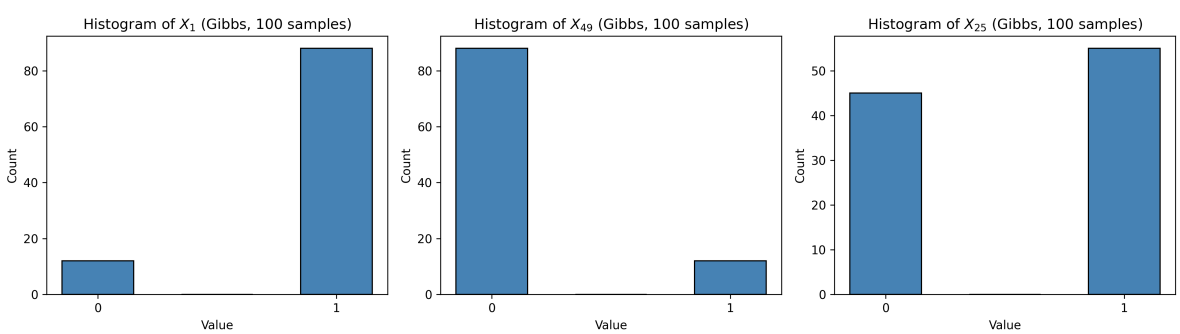
$$P(x_i = 1 \mid \mathbf{x}_{\mathcal{N}(i)}) = \frac{\exp\left(\sum_{j \in \mathcal{N}(i)} \mathbb{I}[1 = x_j]\right)}{\exp\left(\sum_{j \in \mathcal{N}(i)} \mathbb{I}[1 = x_j]\right) + \exp\left(\sum_{j \in \mathcal{N}(i)} \mathbb{I}[0 = x_j]\right)}$$

This is just a sigmoid of the "vote" from the neighbors. We fix all observed nodes at their given values throughout and only update odd-indexed nodes. After a burn-in period the samples are drawn from the correct conditional distribution.

Part (ii): Histograms

I ran Gibbs sampling with 1000 burn-in iterations and collected 100 samples. Results:

- X_1 (top-left corner, neighbors include observed nodes 2 and 8 which are 1): $\bar{X}_1 \approx 0.88$. Strongly biased toward 1.
- X_{49} (bottom-right corner, neighbors include observed nodes 42 and 48 which are 0): $\bar{X}_{49} \approx 0.12$. Strongly biased toward 0.
- X_{25} (center of the grid, at the boundary between the 1-region and 0-region): $\bar{X}_{25} \approx 0.55$. Roughly balanced with a slight bias toward 1.



These results make sense: the Ising model encourages neighboring nodes to agree, so unobserved nodes near the top (near observed 1s) tend toward 1, those near the bottom (near observed 0s) tend toward 0, and the boundary node X_{25} sits roughly at 50/50.

Part (iii): Is rejection sampling a good idea?

No, rejection sampling is a poor choice for this problem. The idea would be to:

1. Sample from the full Ising prior $p(\mathbf{x})$ over all 49 variables (e.g. via Gibbs on the full unconditioned grid)
2. Reject any sample where the 24 observed even-indexed nodes don't match their required values

The acceptance rate is the prior probability that all 24 even-indexed variables simultaneously take their observed values, $P(\mathbf{x}_{\text{even}} = \text{evidence})$. Empirically this is about 1.2×10^{-4} — I observed 234 acceptances in 2,000,000 proposals. To collect 1000 accepted samples we'd need roughly 8,500,000 proposals from the prior, each requiring multiple Gibbs sweeps over 49 nodes. This is extremely slow compared to the direct Gibbs sampler from part (i) which samples the conditional distribution directly with no rejections.

Part (iv): Rejection sampling estimate of $P(X_1 \mid \text{evidence})$

Despite the low acceptance rate, I ran rejection sampling and collected as many accepted samples as feasible. Each proposal is drawn from the Ising prior via 10 Gibbs sweeps on the full (unconditioned) 49-node grid, then accepted only if all 24 observed nodes match the evidence.

With 2,000,000 proposals and 234 accepted (rate $\approx 1.2 \times 10^{-4}$), the rejection sampling estimate is $\hat{P}(X_1 = 1 \mid \text{evidence}) = 0.842$, consistent with the Gibbs estimate of 0.88 (within $\sim$1.5 standard errors).

Should the estimates be consistent? Yes. Both methods sample from the exact same distribution: $p(\mathbf{x}_{\text{odd}} \mid \mathbf{x}_{\text{even}} = \text{evidence})$. Rejection sampling produces i.i.d. draws (no autocorrelation), while Gibbs samples are correlated across iterations, but as the number of samples grows both converge to the true conditional. The only difference is efficiency: Gibbs generates useful samples at every iteration, while rejection sampling wastes the vast majority of its proposals.

Code:

```
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

np.random.seed(42)

N = 7
num_nodes = N * N

#build adjacency (1 indexed label)
def get_neighbors(label):
    r = (label - 1) // N
    c = (label - 1) % N
    nbrs = []
    if r > 0: nbrs.append((r-1)*N + c + 1)
    if r < N-1: nbrs.append((r+1)*N + c + 1)
    if c > 0: nbrs.append(r*N + c) #c-1 + 1
    if c < N-1: nbrs.append(r*N + c + 2) #c+1 + 1
    return nbrs
```

```
#observed nodes
observed = {}
for i in range(2, 25):
    if i % 2 == 0:
        observed[i] = 1
for i in range(26, 49):
    if i % 2 == 0:
        observed[i] = 0

unobserved = [i for i in range(1,50) if i not in observed]

print(f"observed␣nodes:␣{len(observed)},␣unobserved␣nodes:␣{len(unobserved
   )}")
print(f"observed=1:␣{[k␣for␣k,v␣in␣observed.items()␣if␣v==1]}")
print(f"observed=0:␣{[k␣for␣k,v␣in␣observed.items()␣if␣v==0]}")

#Gibbs sampling
def gibbs_full_conditional(node, state, neighbors):
    nbr_vals = [state[n] for n in neighbors]
    #energy for xi=1: sum I[1 == xj] = number of neighbors equal to 1
    #energy for xi=0: sum I[0 == xj] = number of neighbors equal to 0
    e1 = sum(1 for v in nbr_vals if v == 1)
    e0 = sum(1 for v in nbr_vals if v == 0)
    #p(xi=1) = exp(e1) / (exp(e1) + exp(e0))
    #numerical stability
    max_e = max(e1, e0)
    p1 = np.exp(e1 - max_e) / (np.exp(e1 - max_e) + np.exp(e0 - max_e))
    return p1

def run_gibbs(num_samples, burn_in=500):
    #init state: unobserved nodes random
    state = {}
    for k, v in observed.items():
        state[k] = v
    for node in unobserved:
        state[node] = np.random.randint(0, 2)

    samples = []
    for t in range(burn_in + num_samples):
        for node in unobserved:
            nbrs = get_neighbors(node)
            p1 = gibbs_full_conditional(node, state, nbrs)
            state[node] = 1 if np.random.rand() < p1 else 0

        if t >= burn_in:
            samples.append({k: state[k] for k in unobserved})

    return samples

#part (ii) collect 100 samples
samples = run_gibbs(100, burn_in=1000)

x1_samples = [s[1] for s in samples]
```

```
x49_samples = [s[49] for s in samples]
x25_samples = [s[25] for s in samples]

print(f"X1:  mean={np.mean(x1_samples):.2f}")
print(f"X49: mean={np.mean(x49_samples):.2f}")
print(f"X25: mean={np.mean(x25_samples):.2f}")

fig, axes = plt.subplots(1, 3, figsize=(14, 4))

for ax, data, name in [(axes[0], x1_samples, '$X_1$'),
                       (axes[1], x49_samples, '$X_{49}$'),
                       (axes[2], x25_samples, '$X_{25}$')]:
    ax.hist(data, bins=[-0.25, 0.25, 0.75, 1.25], edgecolor='black',
            rwidth=0.6, color='steelblue')
    ax.set_xticks([0, 1])
    ax.set_xlabel('Value')
    ax.set_ylabel('Count')
    ax.set_title(f'Histogram of {name} (Gibbs, 100 samples)')

plt.tight_layout()
plt.savefig('/Users/skarjagi6/Library/CloudStorage/Dropbox-GaTech/Shreesh
   Karjagi/coursework/GRAPH_ML/HWK3/figs/prob3_gibbs_histograms.png', dpi
   =300, bbox_inches='tight')
plt.close()

#part (iv) rejection sampling

def rejection_sample(num_target, max_tries=2000000):
    #sample from Ising prior, reject if evidence doesn't match
    accepted = []
    total_tries = 0
    while len(accepted) < num_target and total_tries < max_tries:
        total_tries += 1
        #sample from p(x) by Gibbs on the FULL model (no evidence)
        state = {i: np.random.randint(0, 2) for i in range(1, 50)}

        #run a few gibbs sweeps on full model
        for _ in range(10):
            for node in range(1, 50):
                nbrs = get_neighbors(node)
                p1 = gibbs_full_conditional(node, state, nbrs)
                state[node] = 1 if np.random.rand() < p1 else 0

        #check evidence
        match = all(state[k] == v for k, v in observed.items())
        if match:
            accepted.append(state.copy())

        if total_tries % 100000 == 0:
            print(f"  tried {total_tries}, accepted {len(accepted)}")

    return accepted, total_tries

#try rejection sampling
```

```
rej_samples, total_tries = rejection_sample(1000)

acc_rate = len(rej_samples) / total_tries if total_tries > 0 else 0
print(f"Rejection: accepted {len(rej_samples)} / {total_tries} (rate = {
    acc_rate:.2e})")

if len(rej_samples) > 0:
    rej_x1 = [s[1] for s in rej_samples]
    p_x1_rej = np.mean(rej_x1)
    print(f"Rejection estimate: P(X1=1 | evidence) = {p_x1_rej:.4f}")
    print(f"Gibbs estimate:     P(X1=1 | evidence) = {np.mean(x1_samples)
        :.4f}")
    print("These should be consistent -- both sample from the same
        conditional.")
else:
    print("No accepted samples. Rejection sampling is impractical here.")

print("\nDone.")
```

Problem 4

Model: $p(x \mid \mu_1, \mu_2) \propto \frac{1}{2}e^{-(x-\mu_1)^2/2} + \frac{1}{2}e^{-(x-\mu_2)^2/2}$, with prior $p(\mu_1, \mu_2) \propto e^{-\mu_1^2/200}\, e^{-\mu_2^2/200}$.

Part (a): Generate data

I sampled 100 data points from $0.5\mathcal{N}(-5, 1) + 0.5\mathcal{N}(5, 1)$, yielding sample mean ≈ -0.09 and standard deviation ≈ 4.87.

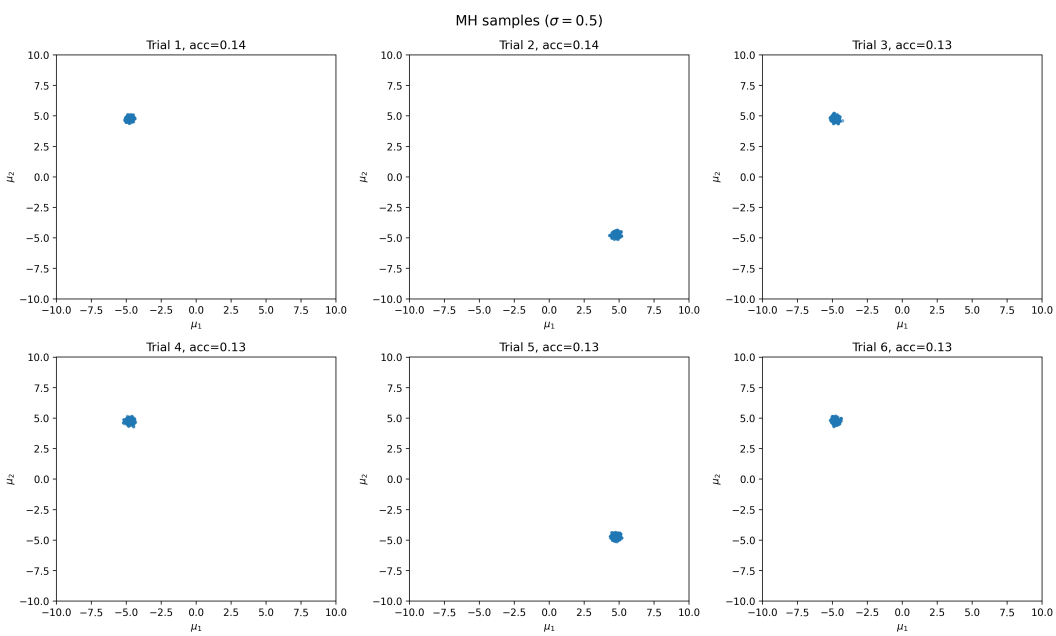
Part (b): Metropolis-Hastings

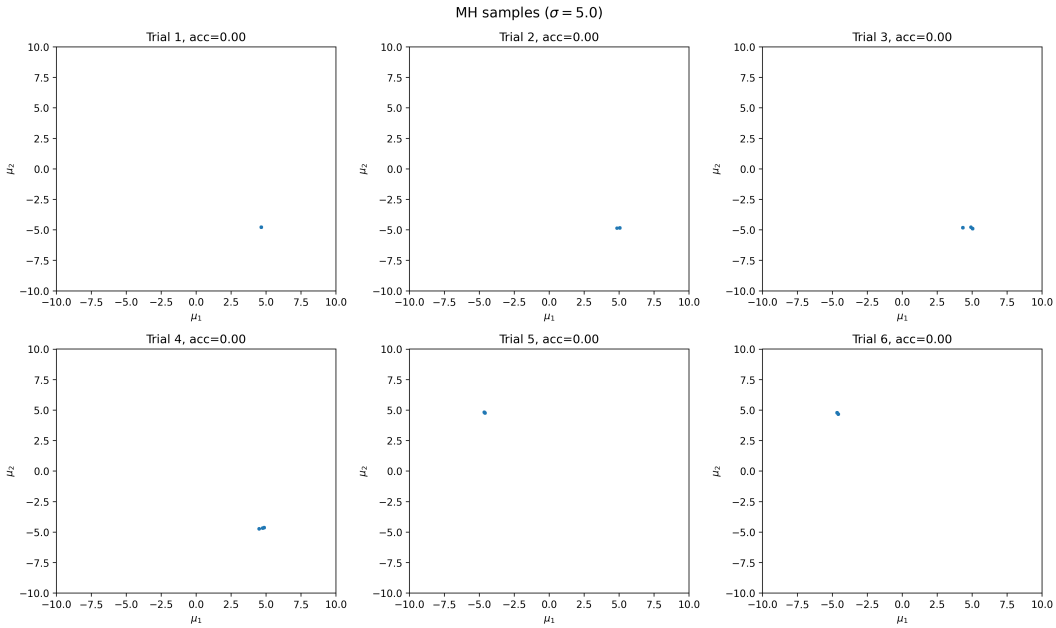
The proposal is $q(\mu_1', \mu_2' \mid \mu_1, \mu_2) = \mathcal{N}(\mu_1', \mu_1, \sigma^2)\mathcal{N}(\mu_2', \mu_2, \sigma^2)$. The acceptance ratio (symmetric proposal) is:

$$A = \min\left(1, \frac{\tilde{p}(\mu_1', \mu_2' \mid \mathbf{x})}{\tilde{p}(\mu_1, \mu_2 \mid \mathbf{x})}\right)$$

where the unnormalized posterior is computed in log-space for stability.

σ	Trial	Acceptance Rate	$\hat{E}[\mu_1]$	$\hat{E}[\mu_2]$
0.5	1–6	$\sim$0.13	$\approx \pm 4.76$	$\approx \mp 4.76$
5.0	1–6	$\sim$0.002	$\approx \pm 4.7$	$\approx \mp 4.7$





Observations:

- $\sigma = 0.5$: acceptance ∼13%, reasonable mixing. The chain explores one mode well but stays stuck in either ($\mu_1 \approx -5, \mu_2 \approx 5$) or its mirror. Different trials land in different modes — this is the label-switching symmetry.
- $\sigma = 5.0$: acceptance ∼0.2%, extremely poor mixing. Large proposals almost always land in low-probability regions and get rejected. The chain barely moves and the scatter plots show tight clumps.

The key difference: $\sigma = 0.5$ makes small steps that land in reasonable regions, while $\sigma = 5$ overshoots badly. Neither explores both modes in 1000 post-burn-in samples since the modes at $(-5, 5)$ and $(5, -5)$ are separated by a low-probability valley.

Part (c): Gibbs sampling

We introduce latent variables $z_i \in \{1, 2\}$ indicating which component generated x_i. The Gibbs sampler alternates:

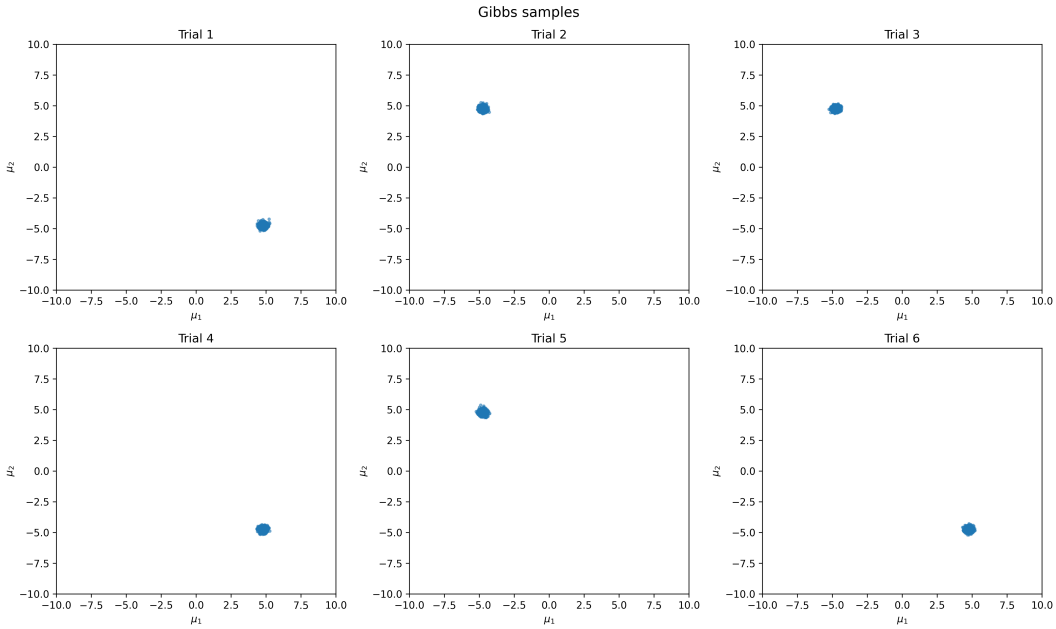
Sample $z_i \mid \mu_1, \mu_2, x_i$:

$$P(z_i = 1 \mid \mu_1, \mu_2, x_i) = \frac{e^{-(x_i-\mu_1)^2/2}}{e^{-(x_i-\mu_1)^2/2} + e^{-(x_i-\mu_2)^2/2}}$$

Sample $\mu_k \mid \mathbf{z}, \mathbf{x}$: Given the cluster assignments, only data points with $z_i = k$ contribute. With prior $\mathcal{N}(0, 100)$ and likelihood $\prod_{i:z_i=k} \mathcal{N}(x_i \mid \mu_k, 1)$, the posterior is:

$$\mu_k \mid \mathbf{z}, \mathbf{x} \sim \mathcal{N}\left(\frac{n_k \bar{x}_k}{n_k + 0.01}, \frac{1}{n_k + 0.01}\right)$$

where $n_k = \#\{i : z_i = k\}$ and $\bar{x}_k$ is the mean of those x_i.



Gibbs consistently estimates $\hat{E}[\mu_1] \approx \pm 4.76$, $\hat{E}[\mu_2] \approx \mp 4.76$ across all 6 trials. It mixes better than MH within a mode because the latent variables break the hard mixture likelihood into easy conjugate updates. Like MH, it still can't jump between the two symmetric modes easily.

Code:

```
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

#part (a) generate data
np.random.seed(0)
N = 100
components = np.random.binomial(1, 0.5, N)
data = np.where(components == 0,
                np.random.normal(-5, 1, N),
                np.random.normal(5, 1, N))
print(f"Data␣n={N},␣mean={data.mean():.2f},␣std={data.std():.2f}")

#log unnormalized posterior
def log_posterior(mu1, mu2, x):
    #prior N(0, 100) for each
    lp = -mu1**2 / 200.0 - mu2**2 / 200.0
    #likelihood  product of 0.5*N(xi|mu1,1) + 0.5*N(xi|mu2,1)
    for xi in x:
        l1 = -0.5*(xi - mu1)**2
        l2 = -0.5*(xi - mu2)**2
        mx = max(l1, l2)
        lp += mx + np.log(0.5*np.exp(l1 - mx) + 0.5*np.exp(l2 - mx))
    return lp

#part (b) Metropolis Hastings
def run_mh(x, sigma_prop, n_burn=10000, n_samples=1000, seed=0):
```

```
    np.random.seed(seed)
    mu1, mu2 = 0.0, 0.0
    lp_curr = log_posterior(mu1, mu2, x)

    total = n_burn + n_samples
    accepts = 0
    samples = []

    for t in range(total):
        mu1_prop = mu1 + np.random.normal(0, sigma_prop)
        mu2_prop = mu2 + np.random.normal(0, sigma_prop)
        lp_prop = log_posterior(mu1_prop, mu2_prop, x)

        log_alpha = lp_prop - lp_curr
        if np.log(np.random.rand()) < log_alpha:
            mu1, mu2 = mu1_prop, mu2_prop
            lp_curr = lp_prop
            accepts += 1

        if t >= n_burn:
            samples.append((mu1, mu2))

    acc_rate = accepts / total
    return np.array(samples), acc_rate

for sigma in [0.5, 5.0]:
    print(f"\nsigma␣=␣{sigma}")
    fig, axes = plt.subplots(2, 3, figsize=(15, 9))

    for trial in range(6):
        samples, acc_rate = run_mh(data, sigma, seed=trial*10)
        mu1_est = samples[:, 0].mean()
        mu2_est = samples[:, 1].mean()
        print(f"trial␣{trial+1}␣acc={acc_rate:.3f},␣"
              f"E[mu1]={mu1_est:.2f},␣E[mu2]={mu2_est:.2f}")

        r, c = trial // 3, trial % 3
        axes[r, c].scatter(samples[:, 0], samples[:, 1], s=6, alpha=0.5)
        axes[r, c].set_xlabel('$\\mu_1$')
        axes[r, c].set_ylabel('$\\mu_2$')
        axes[r, c].set_title(f'Trial␣{trial+1},␣acc={acc_rate:.2f}')
        axes[r, c].set_xlim(-10, 10)
        axes[r, c].set_ylim(-10, 10)

    plt.suptitle(f'MH␣samples␣($\\sigma={sigma}$)', fontsize=14)
    plt.tight_layout()
    plt.savefig(f'/Users/skarjagi6/Library/CloudStorage/Dropbox-GaTech/
       Shreesh␣Karjagi/coursework/GRAPH_ML/HWK3/figs/prob4_mh_sigma{sigma
       }.png',
                dpi=300, bbox_inches='tight')
    plt.close()

#part (c) Gibbs sampling with latent variables
def run_gibbs(x, n_burn=10000, n_samples=1000, seed=0):
```

```
    np.random.seed(seed)
    N = len(x)
    mu1, mu2 = 0.0, 0.0
    z = np.random.randint(0, 2, N) #latent component indicators

    samples = []
    total = n_burn + n_samples

    for t in range(total):
        #sample z_i | mu1, mu2, x_i
        for i in range(N):
            l1 = -0.5*(x[i] - mu1)**2
            l2 = -0.5*(x[i] - mu2)**2
            mx = max(l1, l2)
            p1 = np.exp(l1 - mx) / (np.exp(l1 - mx) + np.exp(l2 - mx))
            z[i] = 0 if np.random.rand() < p1 else 1

        #sample mu1 | z, x (conjugate Gaussian)
        #prior N(0, 100), likelihood prod N(xi|mu1,1) for zi=0
        idx1 = (z == 0)
        n1 = idx1.sum()
        if n1 > 0:
            xbar1 = x[idx1].mean()
        else:
            xbar1 = 0.0
        #posterior precision = n1/1 + 1/100 = n1 + 0.01
        prec1 = n1 + 0.01
        post_mean1 = (n1 * xbar1) / prec1
        post_var1 = 1.0 / prec1
        mu1 = np.random.normal(post_mean1, np.sqrt(post_var1))

        #sample mu2 | z, x
        idx2 = (z == 1)
        n2 = idx2.sum()
        if n2 > 0:
            xbar2 = x[idx2].mean()
        else:
            xbar2 = 0.0
        prec2 = n2 + 0.01
        post_mean2 = (n2 * xbar2) / prec2
        post_var2 = 1.0 / prec2
        mu2 = np.random.normal(post_mean2, np.sqrt(post_var2))

        if t >= n_burn:
            samples.append((mu1, mu2))

    return np.array(samples)

fig, axes = plt.subplots(2, 3, figsize=(15, 9))

for trial in range(6):
    samples = run_gibbs(data, seed=trial*10)
    mu1_est = samples[:, 0].mean()
    mu2_est = samples[:, 1].mean()
```

```
    print(f"trial␣{trial+1}␣E[mu1]={mu1_est:.2f},␣E[mu2]={mu2_est:.2f}")

    r, c = trial // 3, trial % 3
    axes[r, c].scatter(samples[:, 0], samples[:, 1], s=6, alpha=0.5)
    axes[r, c].set_xlabel('$\\mu_1$')
    axes[r, c].set_ylabel('$\\mu_2$')
    axes[r, c].set_title(f'Trial␣{trial+1}')
    axes[r, c].set_xlim(-10, 10)
    axes[r, c].set_ylim(-10, 10)

plt.suptitle('Gibbs␣samples', fontsize=14)
plt.tight_layout()
plt.savefig('/Users/skarjagi6/Library/CloudStorage/Dropbox-GaTech/Shreesh␣
   Karjagi/coursework/GRAPH_ML/HWK3/figs/prob4_gibbs.png', dpi=300,
   bbox_inches='tight')
plt.close()
```

Problem 5 (Blocked Gibbs Sampling for RBM)

The RBM joint distribution is:

$$p(\mathbf{v}, \mathbf{h}) = \frac{1}{Z} \exp\left(-\sum_i b_i v_i - \sum_j c_j h_j - \sum_{i,j} w_{ij} v_i h_j \right)$$

with $v_i, h_j \in \{0, 1\}$.

Part (i): Deriving $p(\mathbf{h} \mid \mathbf{v})$ and $p(\mathbf{v} \mid \mathbf{h})$

Conditional $p(\mathbf{h} \mid \mathbf{v})$:

$$\begin{aligned} p(\mathbf{h} \mid \mathbf{v}) &\propto \exp\left(-\sum_j c_j h_j - \sum_{i,j} w_{ij} v_i h_j \right) \\ &= \prod_j \exp\left(-h_j \left(c_j + \sum_i w_{ij} v_i \right) \right) \end{aligned}$$

This factorizes over j, so conditioned on $\mathbf{v}$, the hidden units h_j are **independent**. For each:

$$P(h_j = 1 \mid \mathbf{v}) = \frac{e^{-(c_j + \sum_i w_{ij} v_i)}}{1 + e^{-(c_j + \sum_i w_{ij} v_i)}} = \sigma\left(-c_j - \sum_i w_{ij} v_i \right)$$

where $\sigma(a) = 1/(1 + e^{-a})$ is the sigmoid.

$$\boxed{p(\mathbf{h} \mid \mathbf{v}) = \prod_j \text{Bernoulli}\Big(h_j \;\Big|\; \sigma(-c_j - \textstyle\sum_i w_{ij} v_i) \Big)}$$

Conditional $p(\mathbf{v} \mid \mathbf{h})$: By the same argument (the graph is bipartite):

$$p(\mathbf{v} \mid \mathbf{h}) \propto \exp\left(-\sum_i b_i v_i - \sum_{i,j} w_{ij} v_i h_j \right) = \prod_i \exp\left(-v_i \left(b_i + \sum_j w_{ij} h_j \right) \right)$$

$$\boxed{p(\mathbf{v} \mid \mathbf{h}) = \prod_i \text{Bernoulli}\Big(v_i \;\Big|\; \sigma\Big(-b_i - \textstyle\sum_j w_{ij} h_j \Big) \Big)}$$

Blocked Gibbs procedure: Since the h_j's are conditionally independent given $\mathbf{v}$ (and vice versa), we can sample all of them in parallel:

1. Given current $\mathbf{v}^{(t)}$, sample **all** $h_j^{(t+1)}$ simultaneously from $p(\mathbf{h} \mid \mathbf{v}^{(t)})$
2. Given $\mathbf{h}^{(t+1)}$, sample **all** $v_i^{(t+1)}$ simultaneously from $p(\mathbf{v} \mid \mathbf{h}^{(t+1)})$

This is valid because the bipartite structure of the RBM means there are no intra-layer connections, making the conditional distributions fully factorized.

Part (ii): Advantage of Blocked Gibbs over Vanilla Gibbs

In vanilla Gibbs sampling on the RBM, we would update one variable at a time: pick a single v_i or h_j, sample from its full conditional, then move to the next. This means each sweep touches $|\mathbf{v}|+|\mathbf{h}|$ variables sequentially, and correlations between variables slow down mixing.

Blocked Gibbs updates *all* hidden units simultaneously in one step and all visible units in the next. This has two advantages:

1. **Faster mixing**: updating an entire layer at once makes larger moves in the joint space per iteration, reducing autocorrelation between successive samples.

2. **Same computational cost per sweep**: each blocked update requires computing $\sigma(-c_j - \sum_i w_{ij} v_i)$ for all j, which is the same total work as doing them one at a time — but we get a much bigger move in one step.

In the context of Contrastive Divergence for training RBMs, this efficiency is crucial: we typically only run 1 Gibbs sweep (CD-1), so making that single sweep as effective as possible is important.

Problem 6 (Pairwise MRF: Loopy BP, Mean Field, Exact)

We have a pairwise MRF on a 2×2 lattice with 4 binary variables (x_1, x_2, x_3, x_4, each taking values $\{1, 2\}$) and 5 potential functions loaded from `pMRF.mat`:

- $\phi_1(x_1, x_2)$, $\phi_2(x_2, x_3)$, $\phi_3(x_3, x_4)$, $\phi_4(x_4, x_1)$, $\phi_5(x_1, x_3)$

  The joint distribution is $p(\mathbf{x}) \propto \prod_{k=1}^{5} \phi_k$.

Part 3: Exact marginals

We enumerate all $2^4 = 16$ configurations, compute the unnormalized probability for each, and normalize. The partition function is $Z = 0.04758$.

Variable	$p(x_i = 1)$	$p(x_i = 2)$
x_1	0.0325	0.9675
x_2	0.7104	0.2896
x_3	0.4509	0.5491
x_4	0.8292	0.1708

Part 1: Loopy BP (factor graph formalism)

I implemented sum-product message passing on the factor graph. The factor graph has 5 factor nodes (one per potential) and 4 variable nodes. Messages alternate between variable→factor and factor→variable updates.

Factor→variable message: For factor f connected to variables $\{v_1, v_2\}$, the message to v_1 is:

$$m_{f \to v_1}(x_{v_1}) = \sum_{x_{v_2}} \phi_f(x_{v_1}, x_{v_2}) \cdot m_{v_2 \to f}(x_{v_2})$$

Variable→factor message: For variable v sending to factor f:

$$m_{v \to f}(x_v) \propto \prod_{g \in \mathrm{ne}(v) \setminus f} m_{g \to v}(x_v)$$

All messages initialized to uniform. BP converged in 14 iterations.

Variable	$q_i^{BP}(x_i = 1)$	$q_i^{BP}(x_i = 2)$
x_1	0.0364	0.9636
x_2	0.7064	0.2936
x_3	0.4510	0.5490
x_4	0.8302	0.1698

Part 2: Variational Mean Field

We approximate $p(\mathbf{x})$ by a fully factorized distribution $q(\mathbf{x}) = \prod_{i=1}^{4} q_i(x_i)$. The mean field update for each q_i is:

$$\log q_i(x_i) = \sum_{f \ni i} \mathbb{E}_{q_{-i}} [\log \phi_f] + \text{const}$$

For a pairwise potential $\phi_f(x_i, x_j)$, this contributes $\sum_{x_j} q_j(x_j) \log \phi_f(x_i, x_j)$ to the update for $q_i(x_i)$.

I iterated coordinate-ascent updates until convergence (29 iterations).

Variable	$q_i^{MF}(x_i = 1)$	$q_i^{MF}(x_i = 2)$
x_1	0.0021	0.9979
x_2	0.9216	0.0784
x_3	0.5668	0.4332
x_4	0.8779	0.1221

Part 4: Comparison

The mean expected deviation is:

$$\Delta = \frac{1}{4} \sum_{i=1}^{4} \frac{1}{2} \sum_{j=1}^{2} |q_i(x_i = j) - p_i(x_i = j)|$$

Method	Mean Deviation Δ
Loopy BP	**0.0022**
Mean Field	0.1016

Comments: BP is dramatically more accurate than MF on this problem, with errors about 50× smaller. Even though the graph has cycles (which means BP is not guaranteed to be exact), the approximation is excellent here — the marginals are within 0.4% of exact. Mean field, on the other hand, struggles because the fully factorized assumption cannot capture the correlations encoded by the 5 pairwise potentials. The biggest MF error is on x_2 (0.921 vs 0.710 exact), where MF overestimates the probability of state 1 by more than 0.2. This makes sense because x_2 participates in three potentials (ϕ_1, ϕ_2, and indirectly through the cycle), and the mean field approximation loses information about how these factors interact.

Code:

```
import numpy as np
import scipy.io

#load the pMRF data
data = scipy.io.loadmat('Data-Problem-6/pMRF.mat')
phi_raw = data['phi'][0]

#parse potential functions
#phi_raw[k] is a tuple (variables_array, table_array)
potentials = []
for k in range(5):
    varr = phi_raw[k][0].flatten() #variables
    table = phi_raw[k][1]  #2x2 table
    potentials.append({'vars': tuple(varr), 'table': table})
    print(f"phi({k+1}): vars={tuple(varr)}, table=\n{table}")

def phi_val(pot, v1, v2):
```

```
    #get potential value for variable assignments v1, v2 in {0,1}
    return pot['table'][v1, v2]

#marginals by enumeration
#enumerate all 2^4 = 16 configurations
exact_joint = np.zeros((2, 2, 2, 2))

for x1 in range(2):
    for x2 in range(2):
        for x3 in range(2):
            for x4 in range(2):
                val = 1.0
                for pot in potentials:
                    v = pot['vars']
                    assign = {1: x1, 2: x2, 3: x3, 4: x4}
                    val *= phi_val(pot, assign[v[0]], assign[v[1]])
                exact_joint[x1, x2, x3, x4] = val

Z = exact_joint.sum()
exact_joint /= Z
print(f"Z␣=␣{Z:.6f}")

exact_marginals = []
for i in range(4):
    axes_to_sum = tuple(j for j in range(4) if j != i)
    marg = exact_joint.sum(axis=axes_to_sum)
    exact_marginals.append(marg)
    print(f"p(x{i+1}=1)␣=␣{marg[0]:.6f},␣p(x{i+1}=2)␣=␣{marg[1]:.6f}")

#part 1 loopy belief propagation/factor graph formalism

#init all to uniform
max_iters = 200
tol = 1e-8

#msg_v2f[(var, factor_idx)] = array of size 2
#msg_f2v[(factor_idx, var)] = array of size 2
msg_v2f = {}
msg_f2v = {}

for fi, pot in enumerate(potentials):
    for v in pot['vars']:
        msg_v2f[(v, fi)] = np.ones(2) / 2.0
        msg_f2v[(fi, v)] = np.ones(2) / 2.0

for iteration in range(max_iters):
    old_f2v = {k: v.copy() for k, v in msg_f2v.items()}
    #update factor-->variable messages
    for fi, pot in enumerate(potentials):
        v1, v2 = pot['vars']
        #message from factor fi to v1 sum over v2 of (pot * msg_v2f[v2-->
            fi])
        incoming_v2 = msg_v2f[(v2, fi)]
        new_msg = np.zeros(2)
```

```
        for a in range(2):
            for b in range(2):
                new_msg[a] += pot['table'][a, b] * incoming_v2[b]
        new_msg /= new_msg.sum()
        msg_f2v[(fi, v1)] = new_msg
        #message from factor fi to v2 sum over v1 of (pot * msg_v2f[v1-->
            fi])
        incoming_v1 = msg_v2f[(v1, fi)]
        new_msg = np.zeros(2)
        for b in range(2):
            for a in range(2):
                new_msg[b] += pot['table'][a, b] * incoming_v1[a]
        new_msg /= new_msg.sum()
        msg_f2v[(fi, v2)] = new_msg
    #update variable-->factor messages
    for fi, pot in enumerate(potentials):
        for v in pot['vars']:
            #msg from v to fi = product of all incoming f2v msg except fi
            msg = np.ones(2)
            for fj, pot2 in enumerate(potentials):
                if fj == fi:
                    continue
                if v in pot2['vars']:
                    msg *= msg_f2v[(fj, v)]
            msg /= msg.sum()
            msg_v2f[(v, fi)] = msg
    #check convergence
    max_diff = 0
    for k in msg_f2v:
        max_diff = max(max_diff, np.max(np.abs(msg_f2v[k] - old_f2v[k])))

    if max_diff < tol:
        print(f"BP␣converged␣at␣iteration␣{iteration+1}")
        break

if max_diff >= tol:
    print(f"BP␣did␣not␣converge␣after␣{max_iters}␣iters␣(max_diff={
        max_diff:.2e})")

#compute BP beliefs
bp_marginals = []
for vi in range(1, 5):
    belief = np.ones(2)
    for fi, pot in enumerate(potentials):
        if vi in pot['vars']:
            belief *= msg_f2v[(fi, vi)]
    belief /= belief.sum()
    bp_marginals.append(belief)
    print(f"q_BP(x{vi}=1)␣=␣{belief[0]:.6f},␣q_BP(x{vi}=2)␣=␣{belief[1]:.6
        f}")

#part 2 mean field

q = [np.ones(2) / 2.0 for _ in range(4)]  #q[0] = q(x1), etc.
```

```
for iteration in range(500):
    old_q = [qi.copy() for qi in q]

    for vi in range(4): #update q[vi] for variable vi+1
        log_q = np.zeros(2)

        for fi, pot in enumerate(potentials):
            v1, v2 = pot['vars']
            if vi + 1 == v1:
                #factor involves vi+1 as first var other var is v2
                other_idx = v2 - 1  #0 indexed
                for a in range(2):
                    for b in range(2):
                        log_q[a] += q[other_idx][b] * np.log(pot['table'][
                            a, b] + 1e-300)
            elif vi + 1 == v2:
                #factor involves vi+1 as second var, other var is v1
                other_idx = v1 - 1
                for b in range(2):
                    for a in range(2):
                        log_q[b] += q[other_idx][a] * np.log(pot['table'][
                            a, b] + 1e-300)

        #normalize
        log_q -= log_q.max()
        q[vi] = np.exp(log_q)
        q[vi] /= q[vi].sum()

    #check converge
    max_diff = max(np.max(np.abs(q[i] - old_q[i])) for i in range(4))
    if max_diff < 1e-10:
        print(f"MF␣converged␣at␣iteration␣{iteration+1}")
        break

if max_diff >= 1e-10:
    print(f"MF␣did␣not␣converge␣after␣500␣iters␣(max_diff={max_diff:.2e})"
        )

mf_marginals = []
for vi in range(4):
    mf_marginals.append(q[vi])
    print(f"q_MF(x{vi+1}=1)␣=␣{q[vi][0]:.6f},␣q_MF(x{vi+1}=2)␣=␣{q[vi
        ][1]:.6f}")

#part 4 compare
def mean_deviation(approx_marginals, exact_marginals):
    total = 0.0
    for i in range(4):
        for j in range(2):
            total += abs(approx_marginals[i][j] - exact_marginals[i][j])
    return total / (4 * 2)

bp_dev = mean_deviation(bp_marginals, exact_marginals)
```

```
mf_dev = mean_deviation(mf_marginals, exact_marginals)

print(f"BP␣mean␣deviation:␣{bp_dev:.6f}")
print(f"MF␣␣mean␣deviation:␣{mf_dev:.6f}")

print("\n␣more␣comparison:")
print(f"{'Var':>4}␣{'Val':>4}␣{'Exact':>10}␣{'BP':>10}␣{'MF':>10}")
for i in range(4):
    for j in range(2):
        print(f"x{i+1:>2}␣␣{j+1:>3}␣␣{exact_marginals[i][j]:>10.6f}␣"
              f"{bp_marginals[i][j]:>10.6f}␣{mf_marginals[i][j]:>10.6f}")

print("\nDone.")
```